

Simple exercise questions(Day 7)

1. HashMap – Word Frequency (Sorted by Key)

Problem:

Given a list of words, count how many times each word appears. Return a string where each distinct word and its frequency are printed on a new line, sorted in **ascending alphabetical order**. Format: `word: count`.

Input:

A list of strings `words` ($1 \leq \text{length} \leq 1000$). Words contain only lowercase letters.

Output:

String with lines `<word>: <count>` sorted by word.

Example:

Input: `["banana", "apple", "banana", "apple", "cherry"]`

Output:

```
apple: 2
banana: 2
cherry: 1
```

Driver Code (implement `wordFrequency`):

```

import java.util.*;

public class WordFrequency {
    public static String wordFrequency(List<String> words) {
        // TODO: implement
        return "";
    }

    public static void main(String[] args) {
        // Test cases: each element is {input list, expected output string}
        Object[][] tests = {
            {Arrays.asList("banana", "apple", "banana", "apple", "cherry"),
              "apple: 2\nbanana: 2\ncherry: 1"},
            {Arrays.asList("z", "a", "z"),
              "a: 1\nz: 2"},
            {Collections.emptyList(),
              ""},
            {Arrays.asList("single"),
              "single: 1"},
            {Arrays.asList("aaa", "aaa", "aaa", "aaa"),
              "aaa: 4"}
        };

        int passed = 0;
        for (int i = 0; i < tests.length; i++) {
            List<String> input = (List<String>) tests[i][0];
            String expected = (String) tests[i][1];
            String result = wordFrequency(input);
            if (expected.equals(result)) {
                passed++;
                System.out.println("Test " + (i+1) + " passed");
            } else {
                System.out.println("Test " + (i+1) + " FAILED");
                System.out.println("Expected:\n" + expected);
                System.out.println("Got:\n" + result);
            }
        }
        System.out.println(passed + "/" + tests.length + " tests passed");
    }
}

```

2. HashMap – First Non-Repeating Character

Problem:

Find the **first character** (from the left) that appears **only once** in a given string. If all characters repeat or the string is empty, return `'_'`.

Input:

A string s ($0 \leq \text{length} \leq 1000$), may contain any ASCII characters.

Output:

The first non-repeating character, or `'_'`.

Example:

Input: `"swiss"` → Output: `'w'`

Input: `"aabb"` → Output: `'_'`

Driver Code (implement `firstUniqChar`):

```

public class FirstNonRepeating {
    public static char firstUniqChar(String s) {
        // TODO: implement
        return '_';
    }

    public static void main(String[] args) {
        Object[][] tests = {
            {"swiss", 'w'},
            {"aabb", '_'},
            {"", '_'},
            {"a", 'a'},
            {"abacabad", 'c'},
            {"!@!!@", '@'},
            {"aaabbbccc", '_'}
        };
        int passed = 0;
        for (int i = 0; i < tests.length; i++) {
            String input = (String) tests[i][0];
            char expected = (char) tests[i][1];
            char result = firstUniqChar(input);
            if (expected == result) {
                passed++;
                System.out.println("Test " + (i+1) + " passed");
            } else {
                System.out.println("Test " + (i+1) + " FAILED");
                System.out.println("Input: " + input);
                System.out.println("Expected: " + expected + " but got: " + result);
            }
        }
        System.out.println(passed + "/" + tests.length + " tests passed");
    }
}

```

3. HashMap – Two Sum (Return Indices)

Problem:

Given an array of integers `nums` and an integer `target`, return the **indices of the two numbers** such that they add up to `target`. There is **exactly one solution**, and you may not use the same element twice. Return the two indices as a string "`i j`" with `i < j`.

Input:

- `nums`: array of integers ($2 \leq \text{length} \leq 10^4$)
- `target`: integer

Output:

A string "<index1> <index2>" (0-based, smaller index first).

Example:

Input: nums = [2,7,11,15], target = 9 → Output: "0 1"

Driver Code (implement twoSum):

```
import java.util.*;

public class TwoSum {
    public static String twoSum(int[] nums, int target) {
        // TODO: implement
        return "";
    }

    public static void main(String[] args) {
        Object[][] tests = {
            {new int[]{2,7,11,15}, 9, "0 1"},
            {new int[]{3,2,4}, 6, "1 2"},
            {new int[]{3,3}, 6, "0 1"},
            {new int[]{0, -1, 2, -3, 1}, -2, "0 3"},
            {new int[]{1, 2, 3, 4}, 5, "0 3"}
        };
        int passed = 0;
        for (int i = 0; i < tests.length; i++) {
            int[] nums = (int[]) tests[i][0];
            int target = (int) tests[i][1];
            String expected = (String) tests[i][2];
            String result = twoSum(nums, target);
            if (expected.equals(result)) {
                passed++;
                System.out.println("Test " + (i+1) + " passed");
            } else {
                System.out.println("Test " + (i+1) + " FAILED");
                System.out.println("nums: " + Arrays.toString(nums) + " target: " + target);
                System.out.println("Expected: " + expected + " but got: " + result);
            }
        }
        System.out.println(passed + "/" + tests.length + " tests passed");
    }
}
```

4. TreeMap – Range Query Count

Problem:

Maintain a collection of **unique keys** (integers). After inserting all keys, answer queries of the form $[L, R]$ – count how many keys fall in the inclusive range $[L, R]$. Return the answers as a list of integers.

Input:

- `keys`: array of integers ($1 \leq \text{length} \leq 10^4$, all distinct)
- `queries`: 2D array where each row is $[L, R]$ ($L \leq R$)

Output:

A list of integers, each the count for the corresponding query.

Example:

`keys = [5, 1, 9, 3, 7]`, `queries = [[2,6], [1,9]]` → Output: `[2, 5]`

Driver Code (implement `rangeQueryCount`):

```

import java.util.*;

public class TreeMapRangeQuery {
    public static List<Integer> rangeQueryCount(int[] keys, int[][] queries) {
        // TODO: implement (use TreeMap)
        return new ArrayList<>();
    }

    public static void main(String[] args) {
        Object[][] tests = {
            {
                new int[]{5,1,9,3,7},
                new int[][]{{2,6},{1,9}},
                Arrays.asList(2,5)
            },
            {
                new int[]{10},
                new int[][]{{0,10},{11,20}},
                Arrays.asList(1,0)
            },
            {
                new int[]{-5, 0, 5},
                new int[][]{{-10,10}},
                Arrays.asList(3)
            },
            {
                new int[]{1,2,3,4,5},
                new int[][]{{3,3}},
                Arrays.asList(1)
            }
        };
        int passed = 0;
        for (int i = 0; i < tests.length; i++) {
            int[] keys = (int[]) tests[i][0];
            int[][] queries = (int[][]) tests[i][1];
            List<Integer> expected = (List<Integer>) tests[i][2];
            List<Integer> result = rangeQueryCount(keys, queries);
            if (expected.equals(result)) {
                passed++;
                System.out.println("Test " + (i+1) + " passed");
            } else {
                System.out.println("Test " + (i+1) + " FAILED");
                System.out.println("Keys: " + Arrays.toString(keys));
                System.out.println("Expected: " + expected + " but got: " + result);
            }
        }
    }
}

```

```
    }  
    System.out.println(passed + "/" + tests.length + " tests passed");  
}  
}
```

5. HashSet – First Unique Occurrence Order

Problem:

Given a list of integers, return a list that contains each distinct number **in the order they first appear**. Use a `LinkedHashSet` to maintain insertion order.

Input:

List of integers `nums` ($0 \leq \text{length} \leq 1000$).

Output:

List of unique numbers in first-occurrence order.

Example:

Input: [3, 1, 4, 1, 5, 9, 2, 6, 5] → Output: [3, 1, 4, 5, 9, 2, 6]

Driver Code (implement `uniqueInOrder`):

```

import java.util.*;

public class UniqueOrder {
    public static List<Integer> uniqueInOrder(List<Integer> nums) {
        // TODO: implement using LinkedHashSet
        return new ArrayList<>();
    }

    public static void main(String[] args) {
        Object[][] tests = {
            {Arrays.asList(3,1,4,1,5,9,2,6,5), Arrays.asList(3,1,4,5,9,2,6)},
            {Arrays.asList(1,1,1), Arrays.asList(1)},
            {Collections.emptyList(), Collections.emptyList()},
            {Arrays.asList(5,4,3,2,1), Arrays.asList(5,4,3,2,1)},
            {Arrays.asList(10,20,30,20,10), Arrays.asList(10,20,30)}
        };
        int passed = 0;
        for (int i = 0; i < tests.length; i++) {
            List<Integer> input = (List<Integer>) tests[i][0];
            List<Integer> expected = (List<Integer>) tests[i][1];
            List<Integer> result = uniqueInOrder(input);
            if (expected.equals(result)) {
                passed++;
                System.out.println("Test " + (i+1) + " passed");
            } else {
                System.out.println("Test " + (i+1) + " FAILED");
                System.out.println("Input: " + input);
                System.out.println("Expected: " + expected + " but got: " + result);
            }
        }
        System.out.println(passed + "/" + tests.length + " tests passed");
    }
}

```

6. HashSet – Intersection of Two Arrays (Sorted)

Problem:

Compute the **intersection** of two integer arrays. Return the common elements **sorted in ascending order**, each only once.

Use a `HashSet` and then a `TreeSet` (or sort afterwards).

Input:

- `nums1, nums2`: two integer arrays ($0 \leq \text{length} \leq 1000$).

Output:

List of integers (sorted, no duplicates).

Example:

[1,2,2,1] and [2,2] → [2]

[4,9,5] and [9,4,9,8,4] → [4,9]

Driver Code (implement intersectionSorted):

```
import java.util.*;

public class ArrayIntersection {

    public static List<Integer> intersectionSorted(int[] nums1, int[] nums2) {
        // TODO: implement
        return new ArrayList<>();
    }

    public static void main(String[] args) {
        Object[][] tests = {
            {new int[]{1,2,2,1}, new int[]{2,2}, Arrays.asList(2)},
            {new int[]{4,9,5}, new int[]{9,4,9,8,4}, Arrays.asList(4,9)},
            {new int[]{}, new int[]{1,2}, Collections.emptyList()},
            {new int[]{3,1,2}, new int[]{2,3}, Arrays.asList(2,3)},
            {new int[]{1}, new int[]{1}, Arrays.asList(1)}
        };

        int passed = 0;
        for (int i = 0; i < tests.length; i++) {
            int[] a = (int[]) tests[i][0];
            int[] b = (int[]) tests[i][1];
            List<Integer> expected = (List<Integer>) tests[i][2];
            List<Integer> result = intersectionSorted(a, b);
            if (expected.equals(result)) {
                passed++;
                System.out.println("Test " + (i+1) + " passed");
            } else {
                System.out.println("Test " + (i+1) + " FAILED");
                System.out.println("Expected: " + expected + " but got: " + result);
            }
        }
        System.out.println(passed + "/" + tests.length + " tests passed");
    }
}
```

7. TreeSet – Running Maximum After Each Insertion

Problem:

You are processing a stream of distinct integers. After each insertion, print the **current maximum** value. Return a list of the maximums in the order they are produced.

Input:

- `nums`: array of integers (distinct, $1 \leq \text{length} \leq 1000$).

Output:

List of integers, where the i -th element is the maximum after the i -th insertion.

Example:

Input: `[3, 1, 4, 1, 5, 9]` (note: second 1 is ignored because distinct, but will be processed as insertion of a new unique number? We'll treat it as adding all numbers, duplicates allowed? But `TreeSet` will keep only distinct; we can still simulate insertion of every number and ignore duplicates when computing max. However, the problem is simpler: after reading each number, add to `TreeSet` and output the last element. Duplicates won't change max. We'll implement it accordingly.) Let's adjust: numbers are not guaranteed distinct, but we still output max after each step. Example: `[2, 2, 3]` → max after 1st:2, after 2nd:2, after 3rd:3 → `[2, 2, 3]`.

Driver Code (implement `runningMax`):

```

import java.util.*;

public class RunningMax {
    public static List<Integer> runningMax(int[] nums) {
        // TODO: use TreeSet to track elements and output last() each step
        return new ArrayList<>();
    }

    public static void main(String[] args) {
        Object[][] tests = {
            {new int[]{2, 2, 3}, Arrays.asList(2,2,3)},
            {new int[]{5, 3, 8, 1, 8}, Arrays.asList(5,5,8,8,8)},
            {new int[]{10}, Arrays.asList(10)},
            {new int[]{-1, -5, -3}, Arrays.asList(-1,-1,-1)},
            {new int[]{100, 200, 150}, Arrays.asList(100,200,200)}
        };
        int passed = 0;
        for (int i = 0; i < tests.length; i++) {
            int[] nums = (int[]) tests[i][0];
            List<Integer> expected = (List<Integer>) tests[i][1];
            List<Integer> result = runningMax(nums);
            if (expected.equals(result)) {
                passed++;
                System.out.println("Test " + (i+1) + " passed");
            } else {
                System.out.println("Test " + (i+1) + " FAILED");
                System.out.println("Input: " + Arrays.toString(nums));
                System.out.println("Expected: " + expected + " but got: " + result);
            }
        }
        System.out.println(passed + "/" + tests.length + " tests passed");
    }
}

```

8. Comparable – Sort Students by Marks (Desc), Name (Asc)

Problem:

Create a `Student` class with `name` (`String`) and `marks` (`int`). Implement `Comparable<Student>` so that a list of students can be sorted **by marks descending**; if marks are equal, **by name ascending**. Then sort a given list and return a string where each student appears on a new line as `"name: marks"`.

Input:

List of `Student` objects.

Output:

A string with lines "<name>: <marks>" sorted.

Example:

Students: [("Alice",85), ("Bob",92), ("Charlie",85)]

Output:

```
Bob: 92
Alice: 85
Charlie: 85
```

Driver Code (complete compareTo in Student):

```
import java.util.*;

class Student implements Comparable<Student> {
    String name;
    int marks;

    Student(String name, int marks) {
        this.name = name;
        this.marks = marks;
    }

    @Override
    public int compareTo(Student other) {
        // TODO: implement comparison
        return 0;
    }

    @Override
    public String toString() {
        return name + ": " + marks;
    }
}

public class SortStudents {
    public static String sortAndFormat(List<Student> students) {
        Collections.sort(students);
        StringBuilder sb = new StringBuilder();
        for (Student s : students) {
            if (sb.length() > 0) sb.append("\n");
            sb.append(s);
        }
        return sb.toString();
    }

    public static void main(String[] args) {
        List<Student> test1 = Arrays.asList(
            new Student("Alice", 85),
            new Student("Bob", 92),
            new Student("Charlie", 85)
        );
        String expected1 = "Bob: 92\nAlice: 85\nCharlie: 85";

        List<Student> test2 = Arrays.asList(
            new Student("Zoe", 100),
            new Student("Abe", 100),
```

```

        new Student("Bac", 95)
    );
    String expected2 = "Abe: 100\nZoe: 100\nBac: 95";

    List<Student> test3 = Collections.emptyList();
    String expected3 = "";

    Object[][] tests = {
        {test1, expected1},
        {test2, expected2},
        {test3, expected3}
    };
    int passed = 0;
    for (int i = 0; i < tests.length; i++) {
        List<Student> input = (List<Student>) tests[i][0];
        String expected = (String) tests[i][1];
        String result = sortAndFormat(input);
        if (expected.equals(result)) {
            passed++;
            System.out.println("Test " + (i+1) + " passed");
        } else {
            System.out.println("Test " + (i+1) + " FAILED");
            System.out.println("Expected:\n" + expected);
            System.out.println("Got:\n" + result);
        }
    }
    System.out.println(passed + "/" + tests.length + " tests passed");
}
}

```

9. Comparator – Sort Strings by Length then Alphabetically

Problem:

Implement a `Comparator<String>` that sorts strings **by length ascending**, and for equal length, **alphabetically ascending**.

Use it to sort a list and return the sorted strings one per line.

Input:

List of strings.

Output:

String of sorted items separated by newline.

Example:

Input: ["cat", "bird", "dog", "ant"]

Output:

```
ant  
cat  
dog  
bird
```

Driver Code (complete `compare` in `LengthComparator`):

```
import java.util.*;

class LengthComparator implements Comparator<String> {
    @Override
    public int compare(String a, String b) {
        // TODO: implement
        return 0;
    }
}

public class SortByLength {
    public static String sortStrings(List<String> list) {
        list.sort(new LengthComparator());
        return String.join("\n", list);
    }

    public static void main(String[] args) {
        Object[][] tests = {
            {Arrays.asList("cat", "bird", "dog", "ant"), "ant\ncat\ndog\nbird"},
            {Arrays.asList("aaa", "b", "aa"), "b\naa\naaa"},
            {Collections.emptyList(), ""},
            {Arrays.asList("same", "same"), "same\nsame"},
            {Arrays.asList("z", "xy", "abc"), "z\nxy\nabc"}
        };
        int passed = 0;
        for (int i = 0; i < tests.length; i++) {
            List<String> input = (List<String>) tests[i][0];
            String expected = (String) tests[i][1];
            String result = sortStrings(input);
            if (expected.equals(result)) {
                passed++;
                System.out.println("Test " + (i+1) + " passed");
            } else {
                System.out.println("Test " + (i+1) + " FAILED");
                System.out.println("Expected:\n" + expected);
                System.out.println("Got:\n" + result);
            }
        }
        System.out.println(passed + "/" + tests.length + " tests passed");
    }
}
```

10. Combined – Map + Set + Comparable: Frequent Words Sorted by Frequency then Alphabetically

Problem:

Given a list of words, return the **top k** most frequent words. Sort them by **frequency descending**, and for words with the same frequency, **alphabetically ascending**. Output as a string with one word per line. If $k >$ number of distinct words, return all of them sorted.

Input:

- `words`: list of strings
- `k`: integer ($1 \leq k \leq$ number of distinct words)

Output:

String with the top k words, each on a new line.

Example:

`words = ["i", "love", "java", "i", "love", "coding"], k = 2` → Output:

```
i
love
```

Driver Code (implement `topKFrequent`):

```

import java.util.*;

public class TopKFrequentWords {
    public static String topKFrequent(List<String> words, int k) {
        // TODO: use HashMap, then PriorityQueue or sort with custom Comparator
        return "";
    }

    public static void main(String[] args) {
        Object[][] tests = {
            {Arrays.asList("i","love","java","i","love","coding"), 2,
             "i\nlove"},
            {Arrays.asList("the","day","is","sunny","the","the","the","sunny","is","is"), 4,
             "the\nis\nsunny\nday"},
            {Arrays.asList("aaa","aa","a"), 1,
             "a"},
            {Arrays.asList("b","b","a","a"), 2,
             "a\nb"},
            {Arrays.asList("single"), 1,
             "single"}
        };
        int passed = 0;
        for (int i = 0; i < tests.length; i++) {
            List<String> words = (List<String>) tests[i][0];
            int k = (int) tests[i][1];
            String expected = (String) tests[i][2];
            String result = topKFrequent(words, k);
            if (expected.equals(result)) {
                passed++;
                System.out.println("Test " + (i+1) + " passed");
            } else {
                System.out.println("Test " + (i+1) + " FAILED");
                System.out.println("Input: " + words + " k=" + k);
                System.out.println("Expected:\n" + expected);
                System.out.println("Got:\n" + result);
            }
        }
        System.out.println(passed + "/" + tests.length + " tests passed");
    }
}

```

Each driver is self-contained; when you implement the empty method and run the main, you'll immediately see which test cases pass or fail.